14.Exploring Pro-Code Extension of a Zoho Creator Application UsingZoho Catalyst / Custom Functions

Aim

To extend the functional boundaries of a Zoho Creator low-code application by integrating pro-code serverless backend capabilities throughZoho Catalystand customDeluge functions, enabling advanced computation, custom API integrations, and heavy data processing.

Theory

While Zoho Creator provides rapid application development tools (forms, views, workflows, and basic Deluge scripting), complex enterprise requirements often demand pro-code extensions:

1. Custom Deluge Functions:
    a. Concept:User-defined functions written in Deluge (Data Enriched Language for Universal Grid Environment) embedded directly inside Zoho Creator.
    b. Benefit: Encapsulates reusable business logic, triggers automated events on form submissions, and communicates with external REST APIs.
2. Zoho Catalyst Serverless Pro-CodeIntegration:
    a. Concept:Zoho Catalyst is a full-stack serverless pro-code development platform (supporting Java, Node.js, and Python). Zoho Creator applications can offload intensive computing tasks, custom backend

microservices, and heavy database or file operations to Catalyst via APIs or serverless functions.

b. Benefit:Overcomes low-code execution timeouts and computational limitations, granting developers absolute control over source code and backend infrastructure.

Procedure

Step 1: Creating a Custom Deluge Function in Zoho Creator

1. Open your application in theZoho Creator DeveloperConsole.
2. Navigate to the App Setup / Logic section and selectFunctions.
3. Click Create Function, define the function name, return type, and input arguments (e.g.,calculateDiscount(orderTotal decimal, customerTier string)).
4. Write the custom script logic using Deluge, test it using the built-in code executor, and save the function.
5. Associate the custom function with a form workflow event (e.g.,On SuccessorOn Validateof an Order form).

Step 2: Initializing and Setting Up a Zoho Catalyst Project

1. Log in to theZoho Catalyst Consoleand create a newproject.
2. Install the Catalyst CLI locally or use the web-based IDE/VS Code extension to configure a serverless function.
3. Write a custom backend microservice (using Node.js or Python) designed to handle advanced operations (e.g.,

generating complex encrypted PDF reports or running machine learning utility scripts).

Step 3: Connecting Zoho Creator with Zoho Catalyst via API / Integration

1. In Catalyst, deploy the function as anAPI / Basic I/O function to generate an authenticated endpoint URL.
2. Inside Zoho Creator, navigate to Connections under the integration settings and establish a secure connection to your Catalyst project using OAuth/API keys.
3. Write a remote invocation script inside a Zoho Creator custom function usinginvokeurl:

Code snippet

```
response = invokeurl[
    url: "https://api.catalyst.zoho.com/server/project_name/endpoint" type:
    POST
    parameters: {"recordId": input.ID, "amount":input.Total_Amount}
    connection: "catalyst_connection"
];
```

Step 4: Testing and Validating the End-to-End Workflow

1. Submit a test record in the Zoho Creator form to trigger the custom function.
2. Verify that the request successfully payload-transfers to Zoho Catalyst, processes via the pro-code backend function, and returns the computed result back into the Zoho Creator form field or report view in real time.

Result

The Zoho Creator application was successfully extended using pro-code techniques. Custom Deluge functions streamlined internal form automation, while the integration with Zoho Catalyst successfully offloaded complex backend processing tasks to a scalable serverless architecture.